

2. DATABASE STRUCTURE

SAMS (Student Attendance Management System) - Complete Database Design

Table of Contents

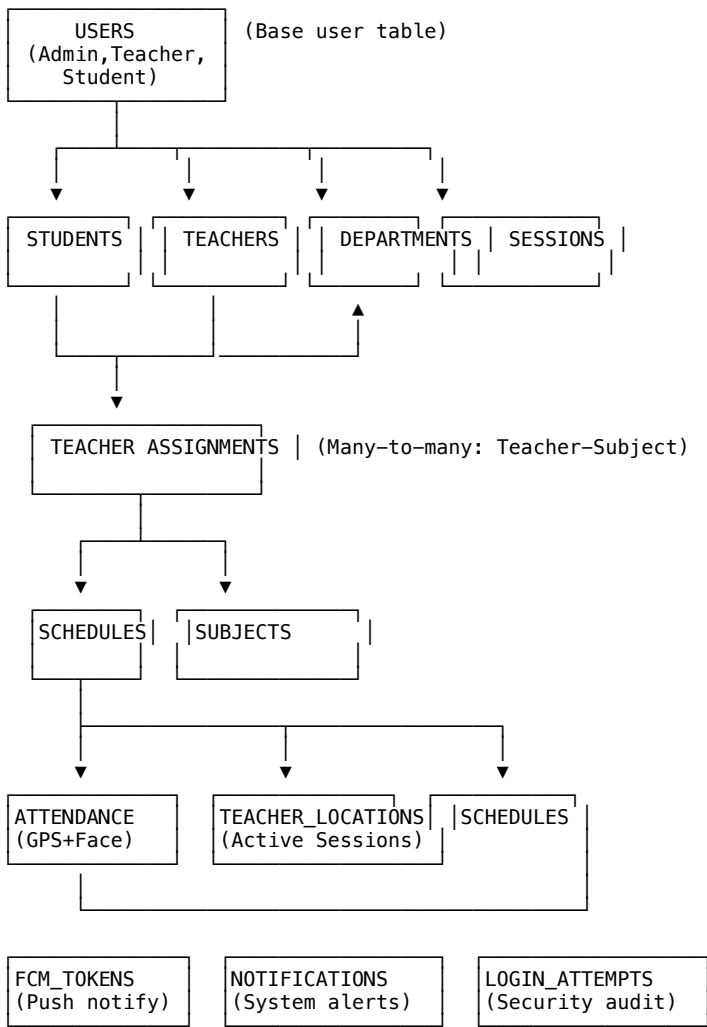
- 1. [Database Overview](#)
- 2. [Entity Relationship Diagram](#)
- 3. [Table Definitions](#)
- 4. [Data Types & Constraints](#)
- 5. [Relationships & Foreign Keys](#)
- 6. [Indexes & Performance](#)
- 7. [Sample Data Flow](#)

Database Overview

Database Name: sams_db **Character Set:** utf8mb4 (supports emoji & special characters) **Collation:** utf8mb4_unicode_ci (case-insensitive) **Engine:** InnoDB (transactions, foreign keys)

Tables: 13 core tables **Total Entities:** Users, Students, Teachers, Departments, Subjects, Schedules, Attendance, FCM Tokens, Notifications, System Settings, Sessions, Audit Logs

Entity Relationship Diagram



AUDIT_LOGS
(Compliance)

SYSTEM_SETTINGS
(Configuration)

Table Definitions

1. USERS Table (Base User Table)

Purpose: Store all users (Admin, Teacher, Student) **Rows:** Auto-increment, ~500-1000 expected

```
CREATE TABLE users (
  id INT PRIMARY KEY AUTO_INCREMENT,
  full_name VARCHAR(100) NOT NULL,
  email VARCHAR(100) UNIQUE NOT NULL,
  password_hash VARCHAR(255) NOT NULL,          -- bcrypt SHA-256
  role ENUM('admin', 'teacher', 'student') NOT NULL,
  phone VARCHAR(15),
  profile_image VARCHAR(255),                  -- URL to profile photo
  is_active BOOLEAN DEFAULT TRUE,              -- Account status
  last_login TIMESTAMP NULL,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,

  INDEX idx_email (email),
  INDEX idx_role (role),
  INDEX idx_active (is_active)
);
```

Columns Explanation: - id: Unique user identifier - email: Login email (unique within system) - password_hash: Hashed password (bcrypt) - role: User type (determines access permissions) - profile_image: URL to profile photo stored in /uploads/ - is_active: Soft delete (account deactivation) - last_login: Track login activity for reports

2. DEPARTMENTS Table

Purpose: Store academic departments **Rows:** 5-20 per institution

```
CREATE TABLE departments (
  id INT PRIMARY KEY AUTO_INCREMENT,
  name VARCHAR(100) NOT NULL,                  -- e.g., "Computer Science"
  code VARCHAR(20) UNIQUE NOT NULL,            -- e.g., "CSE"
  description TEXT,
  hod_id INT,                                  -- Head of Department
  is_active BOOLEAN DEFAULT TRUE,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,

  FOREIGN KEY (hod_id) REFERENCES users(id) ON DELETE SET NULL,
  INDEX idx_code (code),
  INDEX idx_active (is_active)
);
```

3. SUBJECTS Table

Purpose: Store academic subjects/courses **Rows:** 50-200 per institution

```
CREATE TABLE subjects (
  id INT PRIMARY KEY AUTO_INCREMENT,
  name VARCHAR(100) NOT NULL,                  -- e.g., "Data Structures"
  code VARCHAR(20) UNIQUE NOT NULL,            -- e.g., "CS201"
  department_id INT NOT NULL,
  credits INT DEFAULT 3,
  semester INT,                                -- Semester level
  description TEXT,
  is_active BOOLEAN DEFAULT TRUE,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,

  FOREIGN KEY (department_id) REFERENCES departments(id) ON DELETE CASCADE,
  INDEX idx_department (department_id),
  INDEX idx_code (code),
  INDEX idx_semester (semester),
  INDEX idx_active (is_active)
);
```

4. STUDENTS Table

Purpose: Store student-specific information **Rows:** 1000-5000 per institution

```
CREATE TABLE students (
  id INT PRIMARY KEY AUTO_INCREMENT,
  user_id INT UNIQUE NOT NULL,           -- Reference to users table
  roll_number VARCHAR(50) UNIQUE NOT NULL, -- e.g., "CSE-001-2024"
  department_id INT NOT NULL,
  semester INT NOT NULL,
  section VARCHAR(10),                  -- e.g., "A", "B"
  batch_year INT,                       -- e.g., 2024
  admission_date DATE,
  face_registered BOOLEAN DEFAULT FALSE, -- Face registration status
  face_data TEXT,                       -- Encrypted AES-256 face embedding
  face_photo LONGBLOB,                  -- Base64 encoded face photo
  face_registration_date TIMESTAMP NULL,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,

  FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE,
  FOREIGN KEY (department_id) REFERENCES departments(id) ON DELETE CASCADE,
  INDEX idx_roll_number (roll_number),
  INDEX idx_department_semester (department_id, semester),
  INDEX idx_section (section),
  INDEX idx_face_registered (face_registered)
);
```

Face Registration: - face_registered: Boolean flag indicating face is registered - face_data: Encrypted facial embedding (512-dim vector) using AES-256 - face_photo: Base64-encoded profile photo (binary data) - Used for ML Kit face detection in Android app

5. TEACHERS Table

Purpose: Store teacher-specific information **Rows:** 50-200 per institution

```
CREATE TABLE teachers (
  id INT PRIMARY KEY AUTO_INCREMENT,
  user_id INT UNIQUE NOT NULL,
  employee_id VARCHAR(50) UNIQUE NOT NULL, -- e.g., "EMP-001"
  primary_department_id INT,              -- Main department
  designation VARCHAR(100),               -- e.g., "Assistant Professor"
  qualification VARCHAR(255),             -- e.g., "B.Tech, M.Tech, PhD"
  joining_date DATE,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,

  FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE,
  FOREIGN KEY (primary_department_id) REFERENCES departments(id) ON DELETE SET NULL,
  INDEX idx_employee_id (employee_id),
  INDEX idx_department (primary_department_id)
);
```

6. TEACHER_ASSIGNMENTS Table

Purpose: Many-to-many relationship between Teachers and Subjects **Rows:** 200-1000 (depends on courses taught)

```
CREATE TABLE teacher_assignments (
  id INT PRIMARY KEY AUTO_INCREMENT,
  teacher_id INT NOT NULL,
  subject_id INT NOT NULL,
  department_id INT NOT NULL,
  section VARCHAR(10),                  -- e.g., "A", "B"
  academic_year VARCHAR(20) NOT NULL,   -- e.g., "2024-25"
  semester INT NOT NULL,
  is_active BOOLEAN DEFAULT TRUE,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,

  FOREIGN KEY (teacher_id) REFERENCES teachers(id) ON DELETE CASCADE,
  FOREIGN KEY (subject_id) REFERENCES subjects(id) ON DELETE CASCADE,
  FOREIGN KEY (department_id) REFERENCES departments(id) ON DELETE CASCADE,
  UNIQUE KEY unique_assignment (teacher_id, subject_id, department_id, section, semester, academic_year),
  INDEX idx_teacher (teacher_id),
  INDEX idx_subject (subject_id),
  INDEX idx_department (department_id),
  INDEX idx_academic_year (academic_year),
  INDEX idx_active (is_active)
);
```

Purpose: Handles multi-branch support where a teacher teaches same subject in multiple sections/departments

7. SCHEDULES Table

Purpose: Store class timetable **Rows:** 500-2000 (multiple sessions per day)

```
CREATE TABLE schedules (
  id INT PRIMARY KEY AUTO_INCREMENT,
  assignment_id INT NOT NULL,           -- Links to teacher_assignments
  day_of_week ENUM('Monday','Tuesday','Wednesday','Thursday','Friday','Saturday') NOT NULL,
  start_time TIME NOT NULL,             -- e.g., "09:00:00"
  end_time TIME NOT NULL,
  classroom VARCHAR(50),                -- e.g., "101", "Lab-A"
  building VARCHAR(50),                 -- e.g., "Building-A"
  is_active BOOLEAN DEFAULT TRUE,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,

  FOREIGN KEY (assignment_id) REFERENCES teacher_assignments(id) ON DELETE CASCADE,
  INDEX idx_assignment (assignment_id),
  INDEX idx_day_time (day_of_week, start_time),
  INDEX idx_active (is_active)
);
```

Data Format: - One row per time slot - Example: CSE Sem-2, “Data Structures”, Teacher “John”, Monday 9:00-10:00, Building-A, Room 101

8. TEACHER_LOCATIONS Table

Purpose: Store active teacher location during class (GPS coordinates) **Rows:** 100-500 per day (depends on active classes)

```
CREATE TABLE teacher_locations (
  id INT PRIMARY KEY AUTO_INCREMENT,
  teacher_id INT NOT NULL,
  schedule_id INT NOT NULL,
  assignment_id INT NOT NULL,
  department_id INT NOT NULL,
  latitude DECIMAL(10, 8) NOT NULL,     -- e.g., 28.6139 (8 decimal places = ~1.1mm precision)
  longitude DECIMAL(11, 8) NOT NULL,    -- e.g., 77.2090
  session_start TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  session_end TIMESTAMP NULL,           -- NULL = active session
  is_active BOOLEAN DEFAULT TRUE,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,

  FOREIGN KEY (teacher_id) REFERENCES teachers(id) ON DELETE CASCADE,
  FOREIGN KEY (schedule_id) REFERENCES schedules(id) ON DELETE CASCADE,
  FOREIGN KEY (assignment_id) REFERENCES teacher_assignments(id) ON DELETE CASCADE,
  FOREIGN KEY (department_id) REFERENCES departments(id) ON DELETE CASCADE,
  INDEX idx_teacher_active (teacher_id, is_active),
  INDEX idx_schedule (schedule_id),
  INDEX idx_session (session_start, session_end)
);
```

Workflow: 1. Teacher clicks “Start Class” → Create entry with current lat/long 2. Used to verify student proximity (GPS verification) 3. Teacher clicks “End Class” → Set session_end timestamp 4. Deleted when class session ends (optional archival)

9. ATTENDANCE Table (Core Data)

Purpose: Store attendance records for each student **Rows:** 50,000-500,000+ over academic year

```
CREATE TABLE attendance (
  id INT PRIMARY KEY AUTO_INCREMENT,
  student_id INT NOT NULL,
  schedule_id INT NOT NULL,
  assignment_id INT NOT NULL,
  teacher_id INT NOT NULL,
  department_id INT NOT NULL,
  attendance_date DATE NOT NULL,
  attendance_time TIMESTAMP DEFAULT CURRENT_TIMESTAMP,

  -- GPS Verification
  student_latitude DECIMAL(10, 8),
  student_longitude DECIMAL(11, 8),
  teacher_latitude DECIMAL(10, 8),
  teacher_longitude DECIMAL(11, 8),
  distance_meters DECIMAL(8, 2),       -- Calculated distance
);
```

```

-- Face Verification
face_confidence_score DECIMAL(5, 2),      -- 0-100 percentage

-- Verification Result
verification_status ENUM('success', 'gps_failed', 'face_failed', 'both_failed') NOT NULL,
status ENUM('present', 'absent', 'late') DEFAULT 'present',
device_info VARCHAR(255),                  -- Device details for debugging
remarks TEXT,                              -- Admin notes
created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,

FOREIGN KEY (student_id) REFERENCES students(id) ON DELETE CASCADE,
FOREIGN KEY (schedule_id) REFERENCES schedules(id) ON DELETE CASCADE,
FOREIGN KEY (assignment_id) REFERENCES teacher_assignments(id) ON DELETE CASCADE,
FOREIGN KEY (teacher_id) REFERENCES teachers(id) ON DELETE CASCADE,
FOREIGN KEY (department_id) REFERENCES departments(id) ON DELETE CASCADE,

UNIQUE KEY unique_attendance (student_id, schedule_id, attendance_date),
INDEX idx_student (student_id),
INDEX idx_date (attendance_date),
INDEX idx_department (department_id),
INDEX idx_verification (verification_status),
INDEX idx_status (status),
INDEX idx_student_date (student_id, attendance_date)
);

```

Verification Logic:

```

├─ GPS Verification (50m radius threshold)
│  └─ distance_meters < 50 → PASS
│  └─ distance_meters >= 50 → FAIL (gps_failed)
├─ Face Verification (75% confidence threshold)
│  └─ face_confidence_score >= 75 → PASS
│  └─ face_confidence_score < 75 → FAIL (face_failed)
└─ Final Status
   └─ Both PASS → verification_status = "success"
   └─ GPS FAIL → verification_status = "gps_failed"
   └─ Face FAIL → verification_status = "face_failed"
   └─ Both FAIL → verification_status = "both_failed"

```

10. FCM_TOKENS Table

Purpose: Store Firebase Cloud Messaging tokens for push notifications **Rows:** 1000-5000 (multiple devices per user)

```

CREATE TABLE fcm_tokens (
  id INT PRIMARY KEY AUTO_INCREMENT,
  user_id INT NOT NULL,
  token VARCHAR(500) NOT NULL,              -- FCM registration token
  device_type ENUM('android', 'ios', 'web') DEFAULT 'android',
  device_name VARCHAR(100),                 -- e.g., "Samsung Galaxy S21"
  is_active BOOLEAN DEFAULT TRUE,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,

  FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE,
  UNIQUE KEY unique_token (token(255)),
  INDEX idx_user (user_id),
  INDEX idx_active (is_active)
);

```

Workflow: 1. Android app registers FCM token on login 2. Backend stores token with device info 3. Backend uses token to send push notifications 4. Token updated on app update or logout

11. NOTIFICATIONS Table

Purpose: Store system notifications & alerts **Rows:** 10,000-100,000 over academic year

```

CREATE TABLE notifications (
  id INT PRIMARY KEY AUTO_INCREMENT,
  title VARCHAR(255) NOT NULL,
  message TEXT NOT NULL,
  notification_type ENUM('attendance_alert', 'low_attendance', 'system', 'schedule_change', 'face_reregister') NOT NULL,
  target_role ENUM('admin', 'teacher', 'student', 'all'),
  target_user_id INT NULL,                  -- Specific user (if applicable)
  target_department_id INT NULL,            -- All students in dept (if applicable)
  data JSON,                               -- Additional data (e.g., {"student_id": 123})
  is_read BOOLEAN DEFAULT FALSE,
  is_sent BOOLEAN DEFAULT FALSE,

```

```

sent_at TIMESTAMP NULL,
read_at TIMESTAMP NULL,
created_by INT, -- Admin who created notification
created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,

FOREIGN KEY (target_user_id) REFERENCES users(id) ON DELETE CASCADE,
FOREIGN KEY (target_department_id) REFERENCES departments(id) ON DELETE CASCADE,
FOREIGN KEY (created_by) REFERENCES users(id) ON DELETE SET NULL,
INDEX idx_target (target_role, target_user_id),
INDEX idx_sent (is_sent),
INDEX idx_read (is_read),
INDEX idx_type (notification_type)
);

```

Notification Types: - attendance_alert: Student should mark attendance - low_attendance: Student below minimum threshold - schedule_change: Class rescheduled - face_reregister: Face registration expired - system: General system announcements

12. SESSIONS Table

Purpose: Track user sessions for security **Rows:** 100-1000 (active sessions)

```

CREATE TABLE sessions (
  id INT PRIMARY KEY AUTO_INCREMENT,
  user_id INT NOT NULL,
  session_id VARCHAR(100) UNIQUE NOT NULL, -- Random session token
  ip_address VARCHAR(45) NOT NULL, -- IPv4 or IPv6
  user_agent TEXT, -- Browser info
  expires_at TIMESTAMP NOT NULL,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,

  FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE,
  INDEX idx_user_session (user_id, session_id),
  INDEX idx_expires (expires_at)
);

```

13. SYSTEM_SETTINGS Table

Purpose: Store application configuration **Rows:** 10-20 (key-value pairs)

```

CREATE TABLE system_settings (
  id INT PRIMARY KEY AUTO_INCREMENT,
  setting_key VARCHAR(100) UNIQUE NOT NULL,
  setting_value TEXT NOT NULL,
  data_type VARCHAR(20), -- 'int', 'string', 'boolean', 'json'
  description TEXT,
  is_active BOOLEAN DEFAULT TRUE,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,

  INDEX idx_key (setting_key)
);

```

Sample Settings:

gps_proximity_radius	50	GPS threshold in meters
face_confidence_threshold	75	Face matching score %
late_threshold_minutes	15	Minutes late threshold
academic_year	2024-25	Current academic year
semester_current	2	Current active semester
maintenance_mode	false	System maintenance flag
attendance_window_minutes	30	Attendance marking window

14. LOGIN_ATTEMPTS Table

Purpose: Security audit trail **Rows:** 1000-10000 (grows with usage)

```

CREATE TABLE login_attempts (
  id INT PRIMARY KEY AUTO_INCREMENT,
  email VARCHAR(100) NOT NULL,
  ip_address VARCHAR(45) NOT NULL,
  user_agent TEXT,
  attempt_time TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  success BOOLEAN DEFAULT FALSE,

  INDEX idx_email_time (email, attempt_time),
  INDEX idx_ip_time (ip_address, attempt_time)
);

```

15. AUDIT_LOGS Table (Optional)

Purpose: Track all system changes for compliance **Rows:** 10000+ (every create/update/delete)

```
CREATE TABLE audit_logs (
  id INT PRIMARY KEY AUTO_INCREMENT,
  user_id INT,
  action VARCHAR(50),                -- 'CREATE', 'UPDATE', 'DELETE'
  entity_type VARCHAR(50),           -- 'STUDENT', 'ATTENDANCE', etc.
  entity_id INT,
  changes JSON,                      -- { "field": "from_value -> to_value" }
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,

  FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE SET NULL,
  INDEX idx_entity (entity_type, entity_id),
  INDEX idx_time (created_at)
);
```

Data Types & Constraints

Primary Data Types

INT	- User IDs, counts, integers
VARCHAR	- Strings with max length (emails, names)
TEXT	- Long text (descriptions, remarks)
LONGBLOB	- Binary data (face photos)
DATE	- Date only (YYYY-MM-DD)
TIMESTAMP	- Date + Time (auto-managed)
TIME	- Time only (HH:MM:SS)
DECIMAL	- Precise decimals (GPS lat/long, percentages)
BOOLEAN	- True/False (is_active, face_registered)
ENUM	- Predefined values (role, status)
JSON	- Structured data (notification data, settings)

Constraints

Primary Keys: id INT PRIMARY KEY AUTO_INCREMENT - Unique identifier for each row - Auto-increment for simple assignment

Foreign Keys: FOREIGN KEY (column) REFERENCES table(column) - Maintain referential integrity - Cascade delete for related records - Set NULL on delete for optional relationships

Unique Keys: UNIQUE KEY unique_name (columns) - Prevent duplicate data (email, roll_number, token)

Indexes: - Improve query performance on frequently searched columns - Composite indexes for multi-column queries - Avoid over-indexing (impacts INSERT/UPDATE performance)

Relationships & Foreign Keys

One-to-Many Relationships

1. **Users** → **Students** (1:1)
 - One user account has one student profile
 - students.user_id → users.id
2. **Users** → **Teachers** (1:1)
 - One user account has one teacher profile
 - teachers.user_id → users.id
3. **Departments** → **Subjects** (1:N)
 - One department offers many subjects
 - subjects.department_id → departments.id
4. **Students** → **Attendance** (1:N)
 - One student has many attendance records
 - attendance.student_id → students.id
5. **Schedules** → **Attendance** (1:N)
 - One schedule has many attendance records
 - attendance.schedule_id → schedules.id

Many-to-Many Relationships

1. **Teachers** ↔ **Subjects** (N:N via TEACHER_ASSIGNMENTS)

- One teacher teaches many subjects
 - One subject taught by many teachers
 - teacher_assignments.teacher_id → teachers.id
 - teacher_assignments.subject_id → subjects.id
2. **Subjects ↔ Schedules** (N:N via TEACHER_ASSIGNMENTS)
- Subject with different time slots
 - schedules.assignment_id → teacher_assignments.id

Indexes & Performance

Index Strategy

Indexed Columns:

users:

- email (UNIQUE, used in login)
- role (used for permission checks)
- is_active (used for active user filtering)

students:

- roll_number (UNIQUE, student identifier)
- (department_id, semester) - composite for batch queries
- face_registered (for face registration queries)

teachers:

- employee_id (UNIQUE, teacher identifier)
- primary_department_id

teacher_assignments:

- (teacher_id, subject_id, department_id, semester, academic_year)
- academic_year (multi-year support)
- is_active

attendance:

- (student_id, schedule_id, attendance_date) - UNIQUE
- (student_id, attendance_date) - composite for history
- verification_status (filtering)
- status (filtering)

Query Examples with Indexes

```
-- Index: (student_id, attendance_date)
SELECT * FROM attendance
WHERE student_id = 123 AND attendance_date = '2024-01-15'
ORDER BY attendance_time DESC;

-- Index: (teacher_id, is_active)
SELECT * FROM teacher_assignments
WHERE teacher_id = 5 AND is_active = TRUE;

-- Index: (subject_id, semester, academic_year)
SELECT ta.* FROM teacher_assignments ta
WHERE ta.subject_id = 10 AND ta.semester = 2 AND ta.academic_year = '2024-25';
```

Sample Data Flow

Attendance Marking Workflow

Sequence:

1. Student opens app → Dashboard screen
Query:

```
SELECT * FROM schedules
WHERE assignment_id IN (SELECT id FROM teacher_assignments
WHERE subject_id IN (SELECT id FROM subjects
WHERE department_id = student.department_id
AND semester = student.semester))
```
2. Teacher starts class
Insert:

```
INSERT INTO teacher_locations
(teacher_id, schedule_id, assignment_id, latitude, longitude)
VALUES (5, 10, 7, 28.6139, 77.2090)
```
3. Student marks attendance
Insert:

```
INSERT INTO attendance
(student_id, schedule_id, assignment_id, teacher_id, department_id,
```

```
attendance_date, student_latitude, student_longitude,
teacher_latitude, teacher_longitude, distance_meters,
face_confidence_score, verification_status, status)
VALUES (123, 10, 7, 5, 2, '2024-01-15', 28.6140, 77.2091,
28.6139, 77.2090, 12.5, 92.3, 'success', 'present')
```

4. Teacher ends class

Update: UPDATE teacher_locations SET session_end = NOW() WHERE id = 456

Action: Auto-mark absent for students not marked

```
UPDATE attendance SET status = 'absent'
WHERE schedule_id = 10 AND attendance_date = '2024-01-15'
AND student_id NOT IN (SELECT student_id FROM attendance
WHERE schedule_id = 10
AND attendance_date = '2024-01-15')
```

5. Query attendance history

```
Select: SELECT a.*, s.subject_name, t.full_name as teacher_name
FROM attendance a
JOIN schedules sch ON a.schedule_id = sch.id
JOIN teacher_assignments ta ON a.assignment_id = ta.id
JOIN subjects s ON ta.subject_id = s.id
JOIN teachers t ON a.teacher_id = t.id
WHERE a.student_id = 123 AND a.attendance_date BETWEEN '2024-01-01' AND '2024-01-31'
ORDER BY a.attendance_date DESC
```

Summary

Database Design Principles: 1. **Normalization:** Minimized data redundancy 2. **Referential Integrity:** Foreign keys maintain consistency 3.

Performance: Strategic indexing for common queries 4. **Scalability:** Designed to handle 1000s of students 5. **Audit Trail:** Login attempts and audit logs for security 6. **Flexibility:** ENUM types for status values, JSON for extensibility

Key Optimization Strategies: - Composite indexes on frequently combined columns - UNIQUE constraints on identifiers - Pagination for large result sets - Denormalization where necessary (distance_meters in attendance) - Archive tables for old records (optional)